

testrun2.py

```
1 import os
2 import sys
3 import json
4 import time
5 import torch
6 import torch.nn as nn
7 import numpy as np
8 import pandas as pd
9 from pathlib import Path
10 from PIL import Image
11 from torchvision import transforms
12
13 # --- Paths -----
14 SCRIPT_DIR = Path(__file__).resolve().parent
15 MODEL_PATH = SCRIPT_DIR / "report2.pth"
16 DATA_DIR = SCRIPT_DIR / "gen_data_exam"
17 TEST_DIR = DATA_DIR / "test2"
18 OUTPUT_CSV = SCRIPT_DIR / "clsn2_ans.csv"
19 NORM_JSON = SCRIPT_DIR / "outputs" / "norm_stats.json"
20
21 # --- Class mapping -----
22 CLASS_NAMES = ["class 1", "class 2", "class 3", "class 4"]
23 NUM_CLASSES = len(CLASS_NAMES)
24 IN_CHANNELS = 1
25 IMAGE_SIZE = 128
26 BATCH_SIZE = 64
27
28 # --- VGG-16-BN model --
29 VGG16_CFG = [64, 64, "M",
30              128, 128, "M",
31              256, 256, 256, "M",
32              512, 512, 512, "M",
33              512, 512, 512, "M"]
34
35
36 def make_feature_layers(cfg, in_channels):
37     layers = []
38     c = in_channels
39     for v in cfg:
40         if v == "M":
41             layers += [nn.MaxPool2d(kernel_size=2, stride=2)]
42         else:
43             layers += [
44                 nn.Conv2d(c, v, kernel_size=3, padding=1),
45                 nn.BatchNorm2d(v),
46                 nn.ReLU(inplace=True),
47             ]
48             c = v
49     return nn.Sequential(*layers)
50
51
52 class SamVGG16Ex2(nn.Module):
53     """VGG-16 (with BatchNorm) for grayscale 4-class classification."""
54     def __init__(self, num_classes=NUM_CLASSES,
55                 in_channels=IN_CHANNELS, dropout=0.5):
56         super().__init__()
57         self.features = make_feature_layers(VGG16_CFG, in_channels)
58         self.avgpool = nn.AdaptiveAvgPool2d((7, 7))
59         self.classifier = nn.Sequential(
60             nn.Linear(512 * 7 * 7, 4096),
61             nn.ReLU(inplace=True),
62             nn.Dropout(p=dropout),
63             nn.Linear(4096, 4096),
64             nn.ReLU(inplace=True),
65             nn.Dropout(p=dropout),
66             nn.Linear(4096, num_classes),
67         )
68
69     def forward(self, x):
70         x = self.features(x)
```

```

71     x = self.avgpool(x)
72     x = x.flatten(1)
73     return self.classifier(x)
74
75
76 # --- VGG-16 weight table -----
77 def print_weight_table(model):
78     """Print a detailed layer-by-layer parameter count (VGG-16 weight table)."""
79     print("\n" + "=" * 80)
80     print("  VGG-16 LAYER-BY-LAYER WEIGHT TABLE")
81     print("=" * 80)
82     print(f"  {'Layer':<42s} {'Output Shape':<22s} {'Params':>10s}")
83     print(" " + "-" * 76)
84
85     total = 0
86     # Features
87     block, conv_id = 1, 0
88     for i, layer in enumerate(model.features):
89         if isinstance(layer, nn.Conv2d):
90             conv_id += 1
91             w = layer.weight
92             p = sum(pp.numel() for pp in layer.parameters())
93             out_c = w.shape[0]
94             print(f"  Conv2d-{{conv_id:<2d}} (block {{block}}) "
95                   f"3x3, {{w.shape[1]:>3d}}->{{out_c:>3d}}, pad=1"
96                   f"{{':>6s}}{p:>10,d}")
97             total += p
98         elif isinstance(layer, nn.BatchNorm2d):
99             p = sum(pp.numel() for pp in layer.parameters())
100            print(f"  BatchNorm2d (block {{block}})"
101                  f"{{':>26s}}{p:>10,d}")
102            total += p
103        elif isinstance(layer, nn.MaxPool2d):
104            print(f"  MaxPool2d (block {{block}}) "
105                  f"2x2, stride 2{{':>16s}}{'0':>10s}")
106            block += 1
107
108        # AvgPool
109        print(f"  AdaptiveAvgPool2d -> 512x7x7{{':>20s}}{'0':>10s}")
110
111        # Classifier
112        for j, layer in enumerate(model.classifier):
113            if isinstance(layer, nn.Linear):
114                p = sum(pp.numel() for pp in layer.parameters())
115                print(f"  Linear {{layer.in_features:>5d}} -> {{layer.out_features:<5d}}"
116                      f"{{':>22s}}{p:>10,d}")
117                total += p
118            elif isinstance(layer, nn.Dropout):
119                print(f"  Dropout(p={{layer.p}}){{':>28s}}{'0':>10s}")
120
121        print(" " + "-" * 76)
122        print(f"  {'TOTAL':>42s}{{':>22s}}{total:>10,d}")
123        trainable = sum(pp.numel() for pp in model.parameters() if pp.requires_grad)
124        print(f"  {'Trainable':>42s}{{':>22s}}{trainable:>10,d}")
125        print(f"  Model size (FP32):  {{total * 4 / 1e6:.1f}} MB")
126        print("=" * 80)
127
128
129 # --- Main -----
130 def main():
131     # -- 1. Device -----
132     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
133     gpu_name = torch.cuda.get_device_name(0) if device.type == "cuda" else "CPU"
134     print(f"\n Device : {{gpu_name}}")
135
136     # -- 2. Load model -----
137     if not MODEL_PATH.exists():
138         print(f"  [ERROR] Model not found: {{MODEL_PATH}}")
139         print(f"    Ensure report2.pth is next to this script.")
140         sys.exit(1)
141
142     checkpoint = torch.load(MODEL_PATH, map_location=device, weights_only=False)

```

```

143 dropout = checkpoint.get("dropout", 0.5)
144 model = SamVG16Ex2(num_classes=NUM_CLASSES, in_channels=IN_CHANNELS,
145 dropout=dropout).to(device)
146 model.load_state_dict(checkpoint["model_state"])
147 model.eval()
148
149 print(f" Model loaded   : {MODEL_PATH}")
150 print(f" Version       : {checkpoint.get('version', '?')}")
151 print(f" Trained epoch   : {checkpoint.get('epoch', '?')}")
152 print(f" Val accuracy    : {checkpoint.get('val_acc', '?')}")
153 print(f" Val F1 (macro)  : {checkpoint.get('val_f1_macro', '?')}")
154 print(f" Image size      : {checkpoint.get('image_size', IMAGE_SIZE)}")
155 print(f" Input channels   : {checkpoint.get('in_channels', IN_CHANNELS)}")
156 print(f" Seed           : {checkpoint.get('seed', '?')}")
157 print(f" Classes         : {checkpoint.get('class_names', CLASS_NAMES)}")
158
159 # -- 3. Print VGG-16 weight table -----
160 print_weight_table(model)
161
162 # -- 4. Load normalization stats -----
163 if not NORM_JSON.exists():
164     print(f" [WARNING] {NORM_JSON} not found, using fallback stats.")
165     mean_val, std_val = 0.0992, 0.2109
166 else:
167     with open(NORM_JSON, "r", encoding="utf-8") as f:
168         stats = json.load(f)
169         mean_val, std_val = stats["mean"], stats["std"]
170 print(f" Norm stats      : mean={mean_val:.4f} std={std_val:.4f}")
171
172 # -- 5. Build CLEAN test transform (NO noise/jitter -- per professor) --
173 test_transform = transforms.Compose([
174     transforms.Grayscale(num_output_channels=1),
175     transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
176     transforms.ToTensor(),
177     transforms.Normalize(mean=[mean_val], std=[std_val]),
178 ])
179
180 # -- 6. Collect test images -----
181 if not TEST_DIR.exists():
182     print(f" [ERROR] Test folder not found: {TEST_DIR}")
183     sys.exit(1)
184
185 image_files = sorted(
186     f for f in os.listdir(TEST_DIR)
187     if f.lower().endswith((".png", ".jpg", ".jpeg", ".bmp"))
188 )
189 print(f" Test images     : {len(image_files)}")
190
191 if len(image_files) == 0:
192     print(f" [ERROR] No images found in test2 folder.")
193     sys.exit(1)
194
195 # -- 7. Inference -----
196 results = []
197 use_amp = (device.type == "cuda")
198 latencies = []
199
200 print(f"\n Running inference on {len(image_files)} images ...")
201
202 with torch.no_grad():
203     for fname in image_files:
204         img_path = TEST_DIR / fname
205         image = Image.open(img_path).convert("L") # force grayscale
206         tensor = test_transform(image).unsqueeze(0).to(device)
207
208         if device.type == "cuda":
209             torch.cuda.synchronize()
210             t0 = time.perf_counter()
211
212         with torch.amp.autocast(device.type, enabled=use_amp):
213             logits = model(tensor)
214

```

```

215         if device.type == "cuda":
216             torch.cuda.synchronize()
217             latencies.append(time.perf_counter() - t0)
218
219         pred_idx = logits.argmax(dim=1).item()
220         pred_class = CLASS_NAMES[pred_idx]
221         file_stem = Path(fname).stem
222
223         results.append({
224             "filename": file_stem,
225             "prediction": pred_class,
226         })
227
228     # -- 8. Save CSV -----
229     df = pd.DataFrame(results)
230     df.to_csv(OUTPUT_CSV, index=False)
231     print(f"\n Predictions saved -> {OUTPUT_CSV}")
232     print(f" Total predictions : {len(df)}")
233
234     # -- 9. Prediction distribution -----
235     print("\n Prediction distribution:")
236     for cls_name in CLASS_NAMES:
237         count = len(df[df["prediction"] == cls_name])
238         pct = count / len(df) * 100
239         print(f" {cls_name} : {count:>3d} ({pct:.1f}%)"
240
241     # -- 10. Inference statistics -----
242     lat = np.array(latencies) * 1000 # ms
243     print(f"\n Inference speed:")
244     print(f" Mean : {lat.mean():.2f} ms/image")
245     print(f" Median : {np.median(lat):.2f} ms/image")
246     print(f" Std : {lat.std():.2f} ms")
247     print(f" Total : {lat.sum()/1000:.2f} s for {len(image_files)} images")
248     if device.type == "cuda":
249         peak_mb = torch.cuda.max_memory_allocated() / 1e6
250         print(f" Peak VRAM : {peak_mb:.1f} MB")
251
252     # -- 11. CSV preview -----
253     print(f"\n CSV preview (first 10 rows):")
254     print(df.head(10).to_string(index=False))
255
256     print(f"\n Done.\n")
257
258
259 if __name__ == "__main__":
260     main()
261

```